



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления (ИУ)»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии (ИУ7)»

ОТЧЕТ

Лабораторная работа №5

по курсу «Проектирование экспертных систем»

на тему: «Прямая дедукция для определенных выражений методом поиска в
ширину»

Студент ИУ7-32М
(Группа)

(Подпись, дата)

К.Э. Ковалец
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

З.Н. Русакова
(И. О. Фамилия)

2024 г.

Содержание

1	Основная часть	3
1.1	Унификация	3
1.2	Доказательство правила	3
1.3	Алгоритм прямого вывода	4
1.4	Пример работы программы	4
2	Текст программы	8

1 Основная часть

1.1 Унификация

Унификация — это процесс замены предметных переменных аргумента атома другими предметными переменными или константами или функциональными символами. Подстановка называется унификацией, если атомы совпадают.

Унификация выполняется:

- если термы — константы, то они унифицируются, если совпадают;
- если в первом атоме терм — переменная, во втором — константа, то они унифицируются, и переменная получает значение константы;
- если терм в первом атоме — переменная и во втором — переменная, то они унифицируются и становятся связанными, можно использовать любое имя.

Унифицируются между собой термы, стоящие на одинаковых местах в контрарных атомах.

1.2 Доказательство правила

В рамках процесса доказательства правила последовательно рассматриваются входные атомы или входные вершины текущего правила. Чтобы доказать входную вершину нужно найти в базе фактов такой атом, с кроторым будут совпадать имя рассматриваемого предиката и число аргументов, далее необходимо выполнить унификацию пары атомов. Если такой факт найден и унификация прошла успешна, то переменные рассматриваемого атома получают свои значения. Далее значения этих переменных распространяются на остальные атомы данного правила, в которые они входят. После этого переходим к доказательству следующей входной вершины.

При доказательстве атома возможны два варианта: нужный факт в базе присутствует или отсутствует. Если факта нет, то входная вершина, а следовательно и само правило не будут доказаны. Если факт найден и унификация атомов прошла успешна, то полученные значения переменных распространяются по правилу. Если входные вершины доказаны, то доказан и

выходной атом, который записывается в базу фактов или закрытые вершины. Правило считается доказанным и больше не рассматривается.

1.3 Алгоритм прямого вывода

Система дедукции на основе обобщённых правил продукции представляет собой алгоритм, который позволяет делать логические выводы на основе определённых выражений. В основе системы лежит метод резолюции, который позволяет обрабатывать более сложные представления знаний.

Алгоритм можно описать следующим образом:

1. В список закрытых вершин (рабочую память) помещаются атомы, которые являются фактами.
2. Передается целевой атом (возможно, с переменной, возможно, с константой), вводим два флага («решение найдено», «решение не найдено»).
3. Пока оба флага равны *false*:
 - запускается цикл по открытым правилам:
 - проверяется возможность доказательства правила;
 - если оно доказано, то есть входные атомы правила входят в множество фактов, то:
 - * выходной атом правила, переменные в котором уже получили значения, добавляется к фактам (закрытым вершинам);
 - * доказанное правило делается закрытым.

1.4 Пример работы программы

Результаты работы программы приведены в листинге 1.1.

Листинг 1.1 — Результат работы программы

```
1 Текущее правило:
2   W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
3
4 База фактов (закрытые вершины):
5   O(N:const, M:const)
6   M(M:const)
7   A(W:const)
8   E(N:const, A:const)
```

Продолжение листинга 1.1

```
9
10  Список открытых правил:
11      W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
12      M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
13      M(x_2:var) -> W(x_2:var)
14      E(x_3:var, A:const) -> H(x_3:var)
15
16  Список доказанных правил:
17
18  => Пусто
19
20  -----
21
22  Текущее правило:
23      M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
24
25  База фактов (закрытые вершины):
26      O(N:const, M:const)
27      M(M:const)
28      A(W:const)
29      E(N:const, A:const)
30
31  Список открытых правил:
32      W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
33      M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
34      M(x_2:var) -> W(x_2:var)
35      E(x_3:var, A:const) -> H(x_3:var)
36
37  Список доказанных правил:
38
39  => S(W:const, M:const, N:const) с подстановкой {'x_1': M:const}
40
41  -----
42
43  Текущее правило:
44      M(x_2:var) -> W(x_2:var)
45
46  База фактов (закрытые вершины):
47      O(N:const, M:const)
48      M(M:const)
49      A(W:const)
50      E(N:const, A:const)
51      S(W:const, M:const, N:const)
52
53  Список открытых правил:
```

Продолжение листинга 1.1

```
54     W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
55     M(x_2:var) -> W(x_2:var)
56     E(x_3:var, A:const) -> H(x_3:var)
57
58     Список доказанных правил:
59     M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
60
61     => W(M:const) с подстановкой {'x_2': M:const}
62
63     -----
64
65     Текущее правило:
66     E(x_3:var, A:const) -> H(x_3:var)
67
68     База фактов (закрытые вершины):
69     O(N:const, M:const)
70     M(M:const)
71     A(W:const)
72     E(N:const, A:const)
73     S(W:const, M:const, N:const)
74     W(M:const)
75
76     Список открытых правил:
77     W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
78     E(x_3:var, A:const) -> H(x_3:var)
79
80     Список доказанных правил:
81     M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
82     M(x_2:var) -> W(x_2:var)
83
84     => H(N:const) с подстановкой {'x_3': N:const}
85
86     -----
87
88     Текущее правило:
89     W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
90
91     База фактов (закрытые вершины):
92     O(N:const, M:const)
93     M(M:const)
94     A(W:const)
95     E(N:const, A:const)
96     S(W:const, M:const, N:const)
97     W(M:const)
98     H(N:const)
```

Продолжение листинга 1.1

```
99
100  Список открытых правил:
101      W(y:var) & A(x:var) & S(x:var, y:var, z:var) & H(z:var) -> C(x:var)
102
103  Список доказанных правил:
104      M(x_1:var) & O(N:const, x_1:var) -> S(W:const, x_1:var, N:const)
105      M(x_2:var) -> W(x_2:var)
106      E(x_3:var, A:const) -> H(x_3:var)
107
108  => C(W:const) с подстановкой {'y': M:const, 'x': W:const, 'z': N:const}
109
110  -----
111
112  Цель успешно доказана
```

2 Текст программы

В листингах 2.1–2.5 представлен код программы.

Листинг 2.1 — Основной модуль программы

```
1  from deduction import DirectDeduction
2  from utils import parse_atom, parse_knowledge
3
4
5  def get_knowledge():
6      return parse_knowledge(
7          """
8           $O(N, M)$ 
9           $M(M)$ 
10          $A(W)$ 
11          $E(N, A)$ 
12
13          $W(y) \ \& \ A(x) \ \& \ S(x, y, z) \ \& \ H(z) \rightarrow C(x)$ 
14          $M(x_1) \ \& \ O(N, x_1) \rightarrow S(W, x_1, N)$ 
15          $M(x_2) \rightarrow W(x_2)$ 
16          $E(x_3, A) \rightarrow H(x_3)$ 
17         """
18     )
19
20
21  def main() -> None:
22      """
23      Прямая дедукция для определенных выражений методом поиска в ширину
24      """
25      deduction = DirectDeduction(
26          knowledge=get_knowledge(),
27      )
28      if deduction.knowledge_prove(
29          goal=parse_atom("C(W)"),
30      ):
31          print("\nЦель успешно доказана\n")
32      else:
33          print("\nНе удалось доказать цель\n")
34
35
36  if __name__ == "__main__":
37      main()
```


Листинг 2.2 — Модуль с описанием классов для термов, атомов, конъюнктов, импликаций и знаний

```
1  class Term:
2      def __init__(self, name: str, type: str, value=None):
3          self.name = name
4          self.type = type
5          self.value = None
6
7          if type == "const":
8              if value:
9                  self.value = value
10             else:
11                 self.value = name
12
13     def __repr__(self):
14         return "{:}:{:}".format(self.name, self.type)
15
16     def copy(self):
17         return Term(self.name, self.type, self.value)
18
19
20 class Atom:
21     def __init__(self, name: str, args: list[Term], isPositive = True ):
22         self.name = name
23         self.args = args
24         self.isPositive = isPositive
25
26     def __repr__(self):
27         return "{:}({:})".format(
28             '~' if not self.isPositive else '',
29             self.name,
30             ", ".join([
31                 "{:}".format(term) for term in self.args
32             ])
33         )
34
35     def copy(self):
36         return Atom(
37             name=self.name,
38             args=[arg.copy() for arg in self.args],
39             isPositive=self.isPositive
40         )
41
42
43 class Conjunct:
44     def __init__(self, args: list[Atom]):
```

Продолжение листинга 2.2

```
45         self.args = args
46
47     def __repr__(self):
48         if len(self.args) == 0:
49             return "Пycro"
50
51         return " & ".join([
52             "{:}".format(atom) for atom in self.args
53         ])
54
55     def copy(self):
56         return Conjunction(
57             args=[arg.copy() for arg in self.args]
58         )
59
60
61 class Implication:
62     def __init__(self, left: Conjunction, right: Conjunction):
63         self.left = left
64         self.right = right
65
66     def __repr__(self):
67         return "{:} -> {:}".format(self.left, self.right)
68
69     def copy(self):
70         return Implication(
71             left=self.left.copy(),
72             right=self.right.copy()
73         )
74
75
76 class Knowledge:
77     def __init__(
78         self,
79         data: Conjunction,
80         open_rules: list[Implication],
81         close_rules: list[Implication] = [],
82     ):
83         self.data = data
84         self.open_rules = open_rules
85         self.close_rules = close_rules
86
87     def __repr__(self):
88         return "\n".join([
89             "База фактов (закрытые вершины): ",
```

Продолжение листинга 2.2

```
90         *[" {fact}" for fact in self.data.args],
91         "\nСписок открытых правил: ",
92         *[" {rule}" for rule in self.open_rules],
93         "\nСписок доказанных правил: ",
94         *[" {rule}" for rule in self.close_rules],
95     ])
96
97     def copy(self):
98         return Knowledge(
99             data=self.data.copy(),
100             open_rules=[rule.copy() for rule in self.open_rules],
101             close_rules=[rule.copy() for rule in self.close_rules],
102         )
```

Листинг 2.3 — Модуль с реализацией функций унификации и подстановок

```
1  from items import Term, Atom
2
3
4  def add_substitution(substitutions: dict, sub_key: str, term: Term) -> None:
5      for key in substitutions:
6          if substitutions[key].name == sub_key:
7              substitutions[key] = term
8
9      substitutions[sub_key] = term
10
11
12  def add_substitutions(
13      dest_substitutions: dict,
14      source_substitutions: dict,
15  ) -> None:
16      for sub_key in source_substitutions:
17          add_substitution(
18              substitutions=dest_substitutions,
19              sub_key=sub_key,
20              term=source_substitutions[sub_key],
21          )
22
23
24  def unify_atoms(left: Atom, right: Atom) -> dict[str, Term] | None:
25      if left.name != right.name or len(left.args) != len(right.args):
26          return
27
28      substitutions = {}
```

Продолжение листинга 2.3

```
29
30     for i in range(len(left.args)):
31         left_term = left.args[i]
32         right_term = right.args[i]
33
34         if left_term.type == "const" and right_term.type == "const":
35             if left_term.value != right_term.value:
36                 return
37
38         elif left_term.type == "var" and right_term.type == "const":
39             add_substitution(substitutions, left_term.name, right_term)
40
41         elif left_term.type == "const" and right_term.type == "var":
42             add_substitution(substitutions, right_term.name, left_term)
43
44         elif left_term.type == "var" and right_term.type == "var":
45             if left_term.name != right_term.name:
46                 add_substitution(substitutions, left_term.name, right_term)
47
48     return substitutions
49
50
51 def apply_substitution(
52     atoms_list: list[Atom],
53     substitutions: dict[str, Term],
54 ) -> None:
55     for atom in atoms_list:
56         for term in atom.args:
57             if term.name in substitutions:
58                 substitution_term = substitutions[term.name]
59
60                 term.name = substitution_term.name
61                 term.type = substitution_term.type
62
63                 if substitution_term.type == "const":
64                     term.value = substitution_term.value
```

Листинг 2.4 — Модуль с реализацией класса прямой дедукции

```
1     from items import Atom, Conjunct, Implication, Knowledge
2     from unification import unify_atoms, apply_substitution, add_substitutions
3
4
5     class DirectDeduction():
6         def __init__(self, knowledge: Knowledge) -> None:
```

Продолжение листинга 2.4

```
7         self.knowledge = knowledge.copy()
8
9     def knowledge_prove(self, goal: Atom) -> bool:
10         facts = self.knowledge.data
11         open_rules = self.knowledge.open_rules
12         close_rules = self.knowledge.close_rules
13
14         if self.__is_proved(facts.args, goal):
15             return True
16
17         while True:
18             close_rules_count = 0
19
20             i = 0
21             while i < len(open_rules):
22                 print(f"\nТекущее правило:\n {open_rules[i]}")
23
24                 new_fact = self.__try_to_prove_rule(
25                     facts=facts,
26                     rule=open_rules[i],
27                 )
28                 if new_fact:
29                     facts.args += new_fact.args
30                     close_rules.append(open_rules.pop(i))
31                     close_rules_count += 1
32
33                     if self.__is_proved(new_fact.args, goal):
34                         return True
35                 else:
36                     i += 1
37
38             if close_rules_count == 0:
39                 break
40
41         return False
42
43     def __try_to_prove_rule(
44         self,
45         facts: Conjunct,
46         rule: Implication,
47     ) -> Conjunct | None:
48         rule_result = rule.right.copy()
49         rule_input_atoms = rule.left.copy()
50         global_substitutions = {}
51
```

Продолжение листинга 2.4

```
52     while True:
53         unificate_count = 0
54
55         for rule_input_atom in rule_input_atoms.args:
56             for fact_atom in facts.args:
57                 if rule_input_atom.isPositive != fact_atom.isPositive:
58                     continue
59
60                 substitutions = unificate_atoms(
61                     left=rule_input_atom,
62                     right=fact_atom,
63                 )
64                 if substitutions == None:
65                     continue
66
67                 rule_input_atoms.args.remove(rule_input_atom)
68
69                 add_substitutions(
70                     dest_substitutions=global_substitutions,
71                     source_substitutions=substitutions,
72                 )
73                 apply_substitution(
74                     atoms_list=rule_result.args + rule_input_atoms.args,
75                     substitutions=substitutions,
76                 )
77                 unificate_count += 1
78                 break
79
80         if unificate_count == 0:
81             break
82
83     if len(rule_input_atoms.args):
84         print(
85             f"\n{self.knowledge}\n"\
86             f"\n=> Пусто\n"\
87             f"\n{'-' * 64}"
88         )
89         return None
90
91     print(
92         f"\n{self.knowledge}\n"\
93         f"\n=> {rule_result} с подстановкой {global_substitutions}\n"\
94         f"\n{'-' * 64}"
95     )
96     return rule_result
```

Продолжение листинга 2.4

```
97
98     def __is_proved(self, atoms: list[Atom], goal: Atom) -> bool:
99         for atom in atoms:
100             if atom.isPositive != goal.isPositive or \
101                 atom.name != goal.name or \
102                 not self.__equal_types(atom, goal):
103                 continue
104
105             if unificate_atoms(atom, goal) != None:
106                 return True
107
108         return False
109
110     def __equal_types(self, left: Atom, right: Atom) -> bool:
111         if len(left.args) != len(right.args):
112             return False
113
114         for i in range(len(left.args)):
115             if left.args[i].type != right.args[i].type:
116                 return False
117
118         return True
```

Листинг 2.5 — Модуль с реализацией функций парсеров

```
1     from items import Term, Atom, Conjunct, Implication, Knowledge
2
3
4     def parse_term(exp: str) -> Term:
5         exp = exp.strip()
6         if ':' in exp:
7             name, type = [item.strip() for item in exp.split(':')]
8             return Term(name, type)
9
10        return Term(
11            name=exp,
12            type="const" if exp[0].capitalize() == exp[0] else "var"
13        )
14
15
16    def parse_atom(exp: str) -> Atom:
17        exp = exp.strip()
18
19        is_positive = exp[0] != '~'
20        if not is_positive:
```

Продолжение листинга 2.5

```
21         exp = exp[1:].strip()
22
23         atom_name, atom_args = exp.split('(')
24
25         atom_name = atom_name.strip()
26         atom_args = atom_args.split(' ')[0].strip()
27
28         atom_terms = [
29             parse_term(term) for term in atom_args.split(',')
30         ]
31
32         return Atom(
33             name=atom_name,
34             args=atom_terms,
35             isPositive=is_positive,
36         )
37
38
39 def parse_conjunct(exp: str) -> Conjunction:
40     exp = exp.strip()
41     atoms = [
42         parse_atom(atom) for atom in exp.split("&")
43     ]
44
45     return Conjunction(args=atoms)
46
47
48 def parse_implication(exp: str) -> Implication:
49     exp = exp.strip()
50     left, right = exp.split("->")
51
52     return Implication(
53         left=parse_conjunct(left),
54         right=parse_conjunct(right),
55     )
56
57
58 def parse_knowledge(exp: str) -> Knowledge:
59     data = []
60     rules = []
61
62     for string in exp.splitlines():
63         string = string.strip()
64         if not string:
65             continue
```


Продолжение листинга 2.5

```
66
67     try:
68         impl = parse_implication(string)
69         rules.append(impl)
70     except:
71         atom = parse_atom(string)
72         data.append(atom)
73
74     return Knowledge(
75         data=Conjunct(args=data),
76         open_rules=rules,
77     )
```